KSP Datathon 2026 — Challenge 1 Roadmap

Conversational AI for the KSP Crime Database

Deadline: July 26

Phase 0: Setup (Day 1)

1. **Finalize team on Hack2Skill**
    - Complete the "Create a Team" form (name, description, logo)
    - Add all teammates so they appear on your team dashboard
    - Confirm your challenge selection (Challenge 1) is locked in
2. **Get access to resources**
    - Download/access the crime database/dataset provided by organizers (check your H2S dashboard under "Resources" or "Problem Statement")
    - Join the official Discord/Slack/WhatsApp group for the datathon — announcements and dataset

updates go there
    - Re-watch the "Problem Statement Explainer Session" recording if available

3. **Set up your dev environment**
    - Create a shared GitHub repo
    - Set up a Python virtual environment
    - Decide on your LLM provider (Claude API, OpenAI API, or open-source model) and get API keys sorted

4. **Assign roles**
    - Backend/Text-to-SQL lead
    - Frontend/UI lead
    - Data/database lead
    - Pitch & demo lead (can overlap with above)

Phase 1: Understand the Data (Days 2–3)

1. Load the dataset and inspect:
    - Table structure (cases, FIRs, locations, dates, crime types, status)
    - Data quality issues (missing values, inconsistent formats)
2. Write a **data dictionary** — one page describing every table/column. This will be fed to your LLM later so it understands the schema.
3. List 20–30 **sample questions** an officer might realistically ask, e.g.:

- "How many theft cases were filed in Whitefield last month?"
- "Show open cases assigned to Officer X"
- "What's the most common crime type in Indiranagar this year?"

Phase 2: Build the Core Pipeline (Days 4–8)

1. **Set up the database**
   - Load data into PostgreSQL/MySQL/SQLite (whichever is easiest for your team)
2. **Build the Text-to-SQL layer**
   - Write a system prompt that includes your schema/data dictionary
   - Send natural language questions to the LLM, have it return SQL
   - Test against your 20–30 sample questions; fix failures one by one
3. **Add a RAG layer (recommended)**
   - Store table/column descriptions in a vector DB (Chroma or FAISS)
   - Retrieve relevant schema snippets before generating SQL — improves accuracy a lot if your DB has many tables
4. **Execute + respond**

- Run generated SQL against the DB
- Convert raw results back into a natural language answer
- Handle empty results and errors gracefully ("No matching cases found for that query")

5. **Add guardrails**
    - Block any non-SELECT queries (no UPDATE/DELETE/DROP)
    - Restrict to allowed tables only
    - Log all queries for auditability

Phase 3: Build the Interface (Days 9–10)

1. Build a simple chat UI:
    - **Fast option:** Streamlit or Gradio (good enough for a hackathon demo)
    - **Polished option:** React frontend + FastAPI backend, if you have time
2. Show the chat answer AND the generated SQL query alongside it (builds trust/explainability — judges like this)
3. If time allows, add basic charts for aggregate questions (e.g., a bar chart of crime counts by area)

Phase 4: Differentiators (Days 11–12, optional but valuable)

Pick 1–2 of these if time permits — they meaningfully boost your score:

- **Multilingual support** (Kannada/Hindi queries) — strong fit given KSP context
- **Voice input** (speech-to-text before the NLP pipeline)
- **Role-based access** (different officers see different data scopes)
- **Confidence/uncertainty flagging** when the model isn't sure of the SQL it generated

---

Phase 5: Testing & Polish (Day 13)

1. Run through all sample questions again, end-to-end
2. Stress-test with ambiguous/tricky questions to see how gracefully it fails
3. Fix UI bugs, clean up error messages
4. Make sure the whole thing runs from a clean setup (test on a fresh machine/environment if possible)

Phase 6: Prepare Submission Materials (Day 14)

You'll typically need to submit:

1. **Working prototype** — hosted link (Streamlit Cloud, Vercel, Render) or a clear local setup + demo video
2. **GitHub repo** — clean README with setup instructions
3. **Architecture diagram** — simple flow: User → NLP → Text-to-SQL → DB → Response → UI
4. **Demo video (2–3 min)** — show 3–4 real queries working live
5. **Pitch deck (5–8 slides)**:
    - Problem statement
    - Your solution & how it works
    - Tech stack
    - Live demo screenshots
    - Impact for KSP / scalability
    - Team intro

Phase 7: Submit (by July 26)

1. Double-check the exact submission deadline and required format on your H2S dashboard (time zone matters — confirm if it's IST and what cutoff time)
2. Submit well before the deadline — don't wait until the last hour in case of upload issues
3. Confirm submission went through (look for a confirmation email or status change on dashboard)

Quick Checklist

- [ ] Team created on Hack2Skill
- [ ] Dataset reviewed, data dictionary written
- [ ] Sample questions list created
- [ ] Text-to-SQL pipeline working
- [ ] RAG layer added (if time allows)
- [ ] Guardrails in place
- [ ] Chat UI built
- [ ] End-to-end testing done
- [ ] Demo video recorded
- [ ] Pitch deck finished
- [ ] GitHub repo cleaned up with README
- [ ] Final submission made on H2S dashboard